

Alkya + Business Automation: Complete Integration Plan

Transforming Alkya from DevOps Platform to Complete Automation Suite

Executive Summary

Current State: Alkya is a DevOps platform with CI/CD, infrastructure management, and AI capabilities.

Vision: Transform Alkya into a **unified automation platform** that handles BOTH technical operations (DevOps) AND business operations (CRM, email, support, sales).

Market Position: "The only platform where your engineering team and business team use the same AI-powered automation tool"

Timeline: 3-month MVP to market

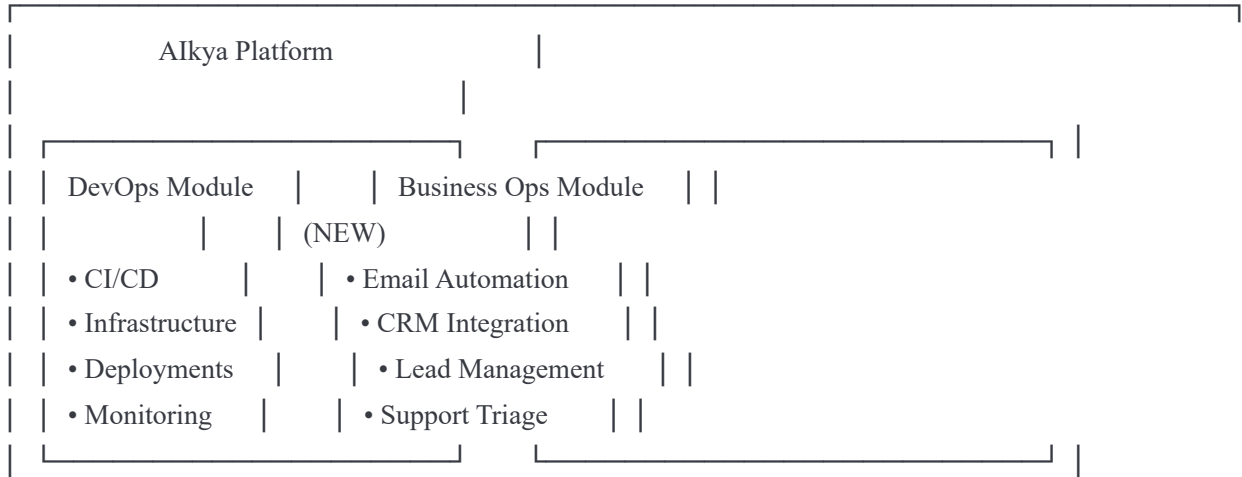
Investment Required: Minimal (leverage existing infrastructure)

Architecture Integration

Current Alkya Stack

Frontend: React + TypeScript + TailwindCSS
Backend: Node.js + Express
Database: SQLite
Auth: JWT + GitHub OAuth
AI: OpenAI, Anthropic, Google AI
Cloud: AWS, GCP, Azure

Proposed Integrated Architecture



Shared Components (Already Exists!)

- AI Engine (OpenAI, Anthropic, Google AI) ✓
- Auth System (JWT + OAuth) ✓
- Database (SQLite) ✓
- API Layer (Express) ✓
- Monitoring & Analytics ✓
- Audit Logging ✓

Key Insight: 80% of the infrastructure you need ALREADY EXISTS in Alkya!

Phase 1: Database Schema Extension (Week 1)

New Tables for Business Automation

sql

-- Add to existing Alkya database (devops_ai.db)

-- Business Automations

```
CREATE TABLE business_automations (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INTEGER NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(50) NOT NULL, -- 'email', 'crm', 'lead', 'support'  
  status VARCHAR(50) DEFAULT 'active',  
  config JSON NOT NULL,  
  triggers JSON,  
  actions JSON,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

-- Business Automation Runs (execution log)

```
CREATE TABLE business_automation_runs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  automation_id INTEGER NOT NULL,  
  status VARCHAR(50), -- 'success', 'failed', 'partial'  
  input_data JSON,  
  output_data JSON,  
  error_message TEXT,  
  duration_ms INTEGER,  
  executed_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (automation_id) REFERENCES business_automations(id)  
);
```

-- Business Leads

```
CREATE TABLE business_leads (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INTEGER NOT NULL,  
  name VARCHAR(255),  
  email VARCHAR(255),  
  phone VARCHAR(50),  
  company VARCHAR(255),  
  budget DECIMAL(10, 2),  
  score INTEGER,  
  status VARCHAR(50) DEFAULT 'new', -- 'new', 'contacted', 'qualified', 'won', 'lost'  
  source VARCHAR(100),  
  notes TEXT,  
  metadata JSON,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Email Log
CREATE TABLE business_emails (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  from_address VARCHAR(255),
  to_address VARCHAR(255),
  subject VARCHAR(500),
  body TEXT,
  classification VARCHAR(50),
  ai_score INTEGER,
  action_taken VARCHAR(100),
  processed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Business Integrations (like Make.com connections)
CREATE TABLE business_integrations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  integration_type VARCHAR(50), -- 'gmail', 'hubspot', 'slack', 'twilio'
  name VARCHAR(255),
  credentials JSON, -- encrypted
  config JSON,
  status VARCHAR(50) DEFAULT 'active',
  last_sync DATETIME,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Create indexes
CREATE INDEX idx_business_automations_user ON business_automations(user_id);
CREATE INDEX idx_business_leads_user ON business_leads(user_id);
CREATE INDEX idx_business_leads_status ON business_leads(status);
CREATE INDEX idx_business_leads_score ON business_leads(score DESC);
CREATE INDEX idx_business_emails_user ON business_emails(user_id);
CREATE INDEX idx_business_automation_runs_automation ON business_automation_runs(automation_id);
```

Phase 2: Backend API Routes (Week 2)

File Structure Addition

```
server/
├── routes/
│   ├── devops/      # Existing DevOps routes
│   │   ├── pipelines.js
│   │   ├── infrastructure.js
│   │   └── deployments.js
│   └── business/    # NEW: Business automation routes
│       ├── automations.js
│       ├── leads.js
│       ├── emails.js
│       ├── integrations.js
│       └── workflows.js
├── services/
│   ├── devops/      # Existing DevOps services
│   └── business/    # NEW: Business automation services
│       ├── aiService.js    # Reuse existing AI service
│       ├── emailService.js
│       ├── crmService.js
│       ├── leadService.js
│       └── workflowEngine.js
└── models/
    ├── devops/      # Existing models
    └── business/    # NEW: Business models
        ├── Automation.js
        ├── Lead.js
        ├── Email.js
        └── Integration.js
```

Example API Implementation

```
javascript
```

```
// server/routes/business/automations.js
```

```
const express = require('express');
const router = express.Router();
const { authenticateToken } = require('.../middleware/auth');
const AutomationService = require('.../services/business/automationService');
```

```
// Get all automations for user
```

```
router.get('/', authenticateToken, async (req, res) => {
  try {
    const automations = await AutomationService.getAll(req.user.id);
    res.json(automations);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Create new automation
```

```
router.post('/', authenticateToken, async (req, res) => {
  try {
    const automation = await AutomationService.create(req.user.id, req.body);
    res.status(201).json(automation);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Execute automation manually
```

```
router.post('/:id/execute', authenticateToken, async (req, res) => {
  try {
    const result = await AutomationService.execute(
      req.params.id,
      req.user.id,
      req.body
    );
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
// Get automation execution history
```

```
router.get('/:id/runs', authenticateToken, async (req, res) => {
  try {
    const runs = await AutomationService.getRuns(req.params.id, req.user.id);
    res.json(runs);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
    res.status(500).json({ error: error.message });
  }
});
```

```
module.exports = router;
```

```
// server/services/business/automationService.js
```

```
const db = require('../../database');
const AIService = require('../aiService'); // Reuse existing!
const WorkflowEngine = require('./workflowEngine');
```

```
class AutomationService {
  static async create(userId, automationData) {
    const { name, type, triggers, actions, config } = automationData;

    const result = await db.run(
      INSERT INTO business_automations
      (user_id, name, type, config, triggers, actions)
      VALUES (?, ?, ?, ?, ?, ?)
    `, [
      userId,
      name,
      type,
      JSON.stringify(config),
      JSON.stringify(triggers),
      JSON.stringify(actions)
    ]);

    return { id: result.lastID, ...automationData };
  }
}
```

```
static async execute(automationId, userId, inputData) {
  const startTime = Date.now();

  try {
    // Get automation
    const automation = await db.get(
      SELECT * FROM business_automations
      WHERE id = ? AND user_id = ?
    `, [automationId, userId]);

    if (!automation) {
      throw new Error('Automation not found');
    }
  }
```

```
// Execute workflow
```

```
const result = await WorkflowEngine.execute({
```

```

    triggers: JSON.parse(automation.triggers),
    actions: JSON.parse(automation.actions),
    config: JSON.parse(automation.config),
    inputData
  });

  // Log execution
  await db.run(`
    INSERT INTO business_automation_runs
    (automation_id, status, input_data, output_data, duration_ms)
    VALUES (?, ?, ?, ?, ?)
  `, [
    automationId,
    'success',
    JSON.stringify(inputData),
    JSON.stringify(result),
    Date.now() - startTime
  ]);

  return result;
} catch (error) {
  // Log failure
  await db.run(`
    INSERT INTO business_automation_runs
    (automation_id, status, input_data, error_message, duration_ms)
    VALUES (?, ?, ?, ?, ?)
  `, [
    automationId,
    'failed',
    JSON.stringify(inputData),
    error.message,
    Date.now() - startTime
  ]);

  throw error;
}

static async getAll(userId) {
  return await db.all(`
    SELECT * FROM business_automations
    WHERE user_id = ?
    ORDER BY created_at DESC
  `, [userId]);
}

static async getRuns(automationId, userId) {

```



```

return await db.all(`
  SELECT r.* FROM business_automation_runs r
  JOIN business_automations a ON r.automation_id = a.id
  WHERE a.id = ? AND a.user_id = ?
  ORDER BY r.executed_at DESC
  LIMIT 100
`, [automationId, userId]);
}
}

module.exports = AutomationService;

// server/services/business/workflowEngine.js
const AIService = require('./aiService'); // Reuse existing AI service
const EmailService = require('./emailService');
const CRMService = require('./crmService');

class WorkflowEngine {
  static async execute(workflow) {
    const { triggers, actions, config, inputData } = workflow;
    let context = { ...inputData };

    // Execute actions in sequence
    for (const action of actions) {
      try {
        const result = await this.executeAction(action, context, config);
        context = { ...context, ...result };
      } catch (error) {
        console.error(`Action failed: ${action.type}`, error);
        throw error;
      }
    }

    return context;
  }

  static async executeAction(action, context, config) {
    switch (action.type) {
      case 'ai_classify':
        return await this.aiClassify(action, context);

      case 'ai_score':
        return await this.aiScore(action, context);

      case 'send_email':
        return await this.sendEmail(action, context, config);
    }
  }
}

```

```

    case 'create_lead':
        return await this.createLead(action, context);

    case 'update_crm':
        return await this.updateCRM(action, context, config);

    case 'send_sms':
        return await this.sendSMS(action, context, config);

    default:
        throw new Error(`Unknown action type: ${action.type}`);
    }
}

static async aiClassify(action, context) {
    const { text, categories } = action.params;

    // Reuse existing AIService from AIkya!
    const prompt = `
Classify this text into one of these categories: ${categories.join(', ')}

Text: ${context[text]}

Return ONLY the category name.
`;

    const response = await AIService.chat(prompt);

    return {
        classification: response.trim(),
        ai_confidence: 0.85 // Could enhance this
    };
}

static async aiScore(action, context) {
    const { data, criteria } = action.params;

    const prompt = `
Score this data from 0-100 based on these criteria:
${criteria}

Data: ${JSON.stringify(context[data])}

Return ONLY a number between 0 and 100.
`;

    const response = await AIService.chat(prompt);

```

```

const score = parseInt(response.trim());

return {
  ai_score: isNaN(score) ? 0 : score
};
}

static async sendEmail(action, context, config) {
  const { to, subject, body } = action.params;

  const resolvedTo = this.resolveValue(to, context);
  const resolvedSubject = this.resolveValue(subject, context);
  const resolvedBody = this.resolveValue(body, context);

  await EmailService.send({
    to: resolvedTo,
    subject: resolvedSubject,
    body: resolvedBody,
    credentials: config.email_credentials
  });

  return { email_sent: true };
}

static async createLead(action, context) {
  const { name, email, company, budget, notes } = action.params;

  const lead = await db.run(`
    INSERT INTO business_leads
    (user_id, name, email, company, budget, notes, status)
    VALUES (?, ?, ?, ?, ?, ?, 'new')
  `, [
    context.user_id,
    this.resolveValue(name, context),
    this.resolveValue(email, context),
    this.resolveValue(company, context),
    this.resolveValue(budget, context),
    this.resolveValue(notes, context)
  ]);

  return { lead_id: lead.lastID };
}

static resolveValue(template, context) {
  // Replace {{variable}} with actual values from context
  return template.replace(/\{\{(\w+)\}\}/g, (match, key) => {
    return context[key] || match;
  });
}

```

```
});  
  
}  
  
}  
  
module.exports = WorkflowEngine;
```

Phase 3: Frontend Integration (Week 3-4)

New React Components

```
src/  
├── pages/  
│   ├── devops/      # Existing DevOps pages  
│   └── business/    # NEW: Business automation pages  
│       ├── Dashboard.tsx  
│       ├── Automations.tsx  
│       ├── AutomationBuilder.tsx  
│       ├── Leads.tsx  
│       ├── EmailManagement.tsx  
│       └── Integrations.tsx  
└── components/  
    ├── devops/      # Existing components  
    └── business/    # NEW: Business components  
        ├── AutomationCard.tsx  
        ├── LeadCard.tsx  
        ├── WorkflowBuilder.tsx  
        ├── EmailClassifier.tsx  
        └── IntegrationSetup.tsx  
└── services/  
    └── api/  
        ├── devops.ts  # Existing  
        └── business.ts # NEW
```

Example Components

```
typescript
```

```
// src/pages/business/Dashboard.tsx
```

```
import React, { useEffect, useState } from 'react';
import { businessAPI } from '../services/api/business';

export default function BusinessDashboard() {
  const [metrics, setMetrics] = useState<any>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadMetrics();
  }, []);

  const loadMetrics = async () => {
    try {
      const data = await businessAPI.getMetrics();
      setMetrics(data);
    } catch (error) {
      console.error('Failed to load metrics:', error);
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <div>Loading...</div>;

  return (
    <div className="p-6">
      <h1 className="text-3xl font-bold mb-6">Business Automation Dashboard</h1>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
        <MetricCard
          title="Emails Processed Today"
          value={metrics.emails_today}
          change="+12%"
          icon="✉️"
        />
        <MetricCard
          title="New Leads"
          value={metrics.new_leads}
          change="+8%"
          icon="🎯"
        />
        <MetricCard
          title="Automations Running"
          value={metrics.active_automations}
          change="+3"
```

```

        icon="⚡ "
      />
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
      <RecentAutomations />
      <HotLeads />
    </div>
  </div>
);
}

// src/pages/business/AutomationBuilder.tsx
import React, { useState } from 'react';
import WorkflowBuilder from '../../components/business/WorkflowBuilder';
import { businessAPI } from '../../services/api/business';

export default function AutomationBuilder() {
  const [workflow, setWorkflow] = useState({
    name: "",
    type: 'email',
    triggers: [],
    actions: []
  });

  const addTrigger = (trigger: any) => {
    setWorkflow(prev => ({
      ...prev,
      triggers: [...prev.triggers, trigger]
    }));
  };

  const addAction = (action: any) => {
    setWorkflow(prev => ({
      ...prev,
      actions: [...prev.actions, action]
    }));
  };

  const saveAutomation = async () => {
    try {
      await businessAPI.createAutomation(workflow);
      alert('Automation created successfully!');
    } catch (error) {
      alert('Failed to create automation');
    }
  };
}

```

```

return (
  <div className="p-6">
    <h1 className="text-3xl font-bold mb-6">Build Automation</h1>

    <div className="bg-white rounded-lg shadow p-6 mb-6">
      <input
        type="text"
        placeholder="Automation Name"
        className="w-full p-3 border rounded mb-4"
        value={workflow.name}
        onChange={(e) => setWorkflow({...workflow, name: e.target.value})}
      />

      <select
        className="w-full p-3 border rounded mb-4"
        value={workflow.type}
        onChange={(e) => setWorkflow({...workflow, type: e.target.value})}
      >
        <option value="email">Email Automation</option>
        <option value="lead">Lead Management</option>
        <option value="support">Support Triage</option>
        <option value="crm">CRM Sync</option>
      </select>
    </div>

    <WorkflowBuilder
      triggers={workflow.triggers}
      actions={workflow.actions}
      onAddTrigger={addTrigger}
      onAddAction={addAction}
    />

    <button
      onClick={saveAutomation}
      className="mt-6 bg-blue-600 text-white px-6 py-3 rounded-lg hover:bg-blue-700"
    >
      Save Automation
    </button>
  </div>
);
}

```

```
// src/components/business/WorkflowBuilder.tsx
```

```
import React from 'react';
```

```
interface Props {
```

```

triggers: any[];
actions: any[];
onAddTrigger: (trigger: any) => void;
onAddAction: (action: any) => void;
}

export default function WorkflowBuilder({ triggers, actions, onAddTrigger, onAddAction }: Props) {
  const actionTemplates = [
    { type: 'ai_classify', label: 'AI Classify', icon: '🤖' },
    { type: 'ai_score', label: 'AI Score', icon: '★' },
    { type: 'send_email', label: 'Send Email', icon: '✉️' },
    { type: 'create_lead', label: 'Create Lead', icon: '🎯' },
    { type: 'send_sms', label: 'Send SMS', icon: '💬' },
    { type: 'update_crm', label: 'Update CRM', icon: '📊' }
  ];

  return (
    <div className="bg-white rounded-lg shadow p-6">
      <h2 className="text-xl font-bold mb-4">Workflow</h2>

      <div className="space-y-4">
        <div className="border-l-4 border-green-500 pl-4">
          <h3 className="font-semibold mb-2">Triggers</h3>
          {triggers.length === 0 ? (
            <p className="text-gray-500">No triggers yet</p>
          ) : (
            triggers.map((trigger, i) => (
              <div key={i} className="bg-gray-50 p-3 rounded mb-2">
                {trigger.type}
              </div>
            ))
          )}
          <button
            onClick={() => onAddTrigger({ type: 'webhook', config: {} })}
            className="text-blue-600 text-sm mt-2"
          >
            + Add Trigger
          </button>
        </div>

        <div className="border-l-4 border-blue-500 pl-4">
          <h3 className="font-semibold mb-2">Actions</h3>
          {actions.length === 0 ? (
            <p className="text-gray-500">No actions yet</p>
          ) : (
            actions.map((action, i) => (
              <div key={i} className="bg-gray-50 p-3 rounded mb-2 flex items-center">

```



```

        <span className="mr-2">{actionTemplates.find(t => t.type === action.type)?.icon}</span>
        {action.type}
      </div>
    ))
  }
}

<div className="grid grid-cols-3 gap-2 mt-3">
  {actionTemplates.map(template => (
    <button
      key={template.type}
      onClick={() => onAddAction({ type: template.type, params: {} })}
      className="bg-blue-50 hover:bg-blue-100 p-2 rounded text-sm"
    >
      {template.icon} {template.label}
    </button>
  ))}
</div>
</div>
</div>
);
}

```

// src/services/api/business.ts

```
import axios from 'axios';
```

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:3001';
```

```
class BusinessAPI {
  async getMetrics() {
    const response = await axios.get(`${API_URL}/api/business/metrics`, {
      headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    });
    return response.data;
  }
}

```

```

async createAutomation(automation: any) {
  const response = await axios.post(
    `${API_URL}/api/business/automations`,
    automation,
    {
      headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    }
  );
  return response.data;
}

```

```

async getAutomations() {
  const response = await axios.get(`${API_URL}/api/business/automations`, {
    headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
  });
  return response.data;
}

async executeAutomation(id: number, data: any) {
  const response = await axios.post(
    `${API_URL}/api/business/automations/${id}/execute`,
    data,
    {
      headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    }
  );
  return response.data;
}

async getLeads() {
  const response = await axios.get(`${API_URL}/api/business/leads`, {
    headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
  });
  return response.data;
}

async createLead(lead: any) {
  const response = await axios.post(
    `${API_URL}/api/business/leads`,
    lead,
    {
      headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    }
  );
  return response.data;
}
}

export const businessAPI = new BusinessAPI();

```

Phase 4: Navigation & UI Integration (Week 4)

Update Main Navigation

typescript

// src/components/Layout/Sidebar.tsx (modify existing)

```
const navigationItems = [
  {
    section: 'DevOps',
    items: [
      { name: 'Pipelines', icon: '🔄', path: '/pipelines' },
      { name: 'Infrastructure', icon: '☁️', path: '/infrastructure' },
      { name: 'Deployments', icon: '🚀', path: '/deployments' },
      { name: 'Monitoring', icon: '📊', path: '/monitoring' }
    ]
  },
  {
    section: 'Business Ops', // NEW SECTION
    items: [
      { name: 'Dashboard', icon: '📈', path: '/business/dashboard' },
      { name: 'Automations', icon: '⚡', path: '/business/automations' },
      { name: 'Leads', icon: '🎯', path: '/business/leads' },
      { name: 'Email Management', icon: '✉️', path: '/business/emails' },
      { name: 'Integrations', icon: '🔌', path: '/business/integrations' }
    ]
  },
  {
    section: 'Settings',
    items: [
      { name: 'Team', icon: '👥', path: '/team' },
      { name: 'Billing', icon: '💰', path: '/billing' },
      { name: 'Profile', icon: '⚙️', path: '/settings' }
    ]
  }
];
```

Update Routing

typescript

```
// src/App.tsx (modify existing)
```

```
import BusinessDashboard from './pages/business/Dashboard';  
import Automations from './pages/business/Automations';  
import AutomationBuilder from './pages/business/AutomationBuilder';  
import Leads from './pages/business/Leads';  
import EmailManagement from './pages/business/EmailManagement';  
import Integrations from './pages/business/Integrations';
```

```
// Add new routes
```

```
<Route path="/business/dashboard" element={<BusinessDashboard />} />  
<Route path="/business/automations" element={<Automations />} />  
<Route path="/business/automations/new" element={<AutomationBuilder />} />  
<Route path="/business/leads" element={<Leads />} />  
<Route path="/business/emails" element={<EmailManagement />} />  
<Route path="/business/integrations" element={<Integrations />} />
```

Phase 5: Cross-Domain Workflows (Week 5-6)

Example: Deploy & Notify Customers

```
javascript
```

```
// server/services/crossDomain/workflows.js
```

```
class CrossDomainWorkflows {
```

```
  // When DevOps deployment succeeds, notify via Business module
```

```
  static async onDeploymentSuccess(deployment) {
```

```
    const { project_name, version, features } = deployment;
```

```
    // Get all customers using Business module
```

```
    const customers = await db.all(`
```

```
      SELECT DISTINCT email FROM business_leads
```

```
      WHERE status = 'customer'
```

```
`);
```

```
    // Use Business email automation
```

```
    for (const customer of customers) {
```

```
      await WorkflowEngine.execute({
```

```
        actions: [{
```

```
          type: 'send_email',
```

```
          params: {
```

```
            to: lead.email,
```

```
            subject: 'Your Personal Demo Environment is Ready!',
```

```
            body: `
```

```
              Hi ${lead.name},
```

Thanks for your interest! We've created a personalized demo environment just for you.

 Demo URL: \${demoEnv.url}

 Username: demo

 Password: \${demoEnv.password}

This demo will be active for 7 days. Explore all the features!

Questions? Just reply to this email.

Best,

Alkya Team

,

```
  }
```

```
}]
```

```
});
```

```
    // Update lead with demo info
```

```
    await db.run(`
```

```
      UPDATE business_leads
```

```
      SET metadata = json_set(
```

```
        COALESCE(metadata, '{}'),
```

```
        '$.demo_url', ?
```

```

    )
    WHERE id = ?
`, [demoEnv.url, lead.id]);
}
}

// When production error occurs, auto-create support ticket
static async onProductionError(error) {
    const { message, stack, affected_users, severity } = error;

    // If customer-impacting, use Business module
    if (affected_users > 0) {
        // Get affected customers
        const customers = await db.all(
            SELECT * FROM business_leads
            WHERE status = 'customer'
            AND id IN (${affected_users.join(',')})
        `);

        // Auto-create support tickets
        for (const customer of customers) {
            await db.run(
                INSERT INTO business_support_tickets
                (customer_id, title, description, priority, status, source)
                VALUES (?, ?, ?, ?, 'open', 'auto_generated')
            `,[
                customer.id,
                `Service Issue: ${message}`,
                `We detected an issue that may have affected your service. Our team is investigating.`,
                severity === 'critical' ? 'urgent' : 'high'
            ]);

            // Send proactive email
            await WorkflowEngine.execute({
                actions: [{
                    type: 'send_email',
                    params: {
                        to: customer.email,
                        subject: 'Service Update: We're On It',
                        body: `
                            Hi ${customer.name},

                            We detected a service issue that may have affected your experience.

                            What happened: ${message}
                            Status: Our team is actively working on it
                            Expected resolution: Within 2 hours
                        `
                    }
                ]
            });
        }
    }
}

```

We've opened a support ticket and will keep you updated.

Ticket ID: #\${customer.id}

Apologies for any inconvenience.

Alkya Team

```
,
}
}]
});
}
}
}
}
}

module.exports = CrossDomainWorkflows;

// Hook into existing DevOps events
// server/services/devops/deploymentService.js (modify existing)
const CrossDomainWorkflows = require('../crossDomain/workflows');

class DeploymentService {
  static async completeDeployment(deploymentId, status) {
    // ... existing deployment completion logic ...

    // NEW: Trigger cross-domain workflow
    if (status === 'success') {
      await CrossDomainWorkflows.onDeploymentSuccess(deployment);
    }

    return deployment;
  }
}

// server/services/business/leadService.js
const CrossDomainWorkflows = require('../crossDomain/workflows');

class LeadService {
  static async create(userId, leadData) {
    // Create lead
    const lead = await db.run(
      INSERT INTO business_leads
      (user_id, name, email, company, budget, score, status)
      VALUES (?, ?, ?, ?, ?, ?, ?)
    , [
```

```
    userId,  
    leadData.name,  
    leadData.email,  
    leadData.company,  
    leadData.budget,  
    leadData.score,  
    'new'  
  ]);  
  
  const newLead = { id: lead.lastID, ...leadData };  
  
  // NEW: If hot lead, trigger cross-domain workflow  
  if (leadData.score >= 80) {  
    await CrossDomainWorkflows.onHotLead(newLead);  
  }  
  
  return newLead;  
}  
}
```

Phase 6: Pricing & Packaging (Week 7)

New Pricing Structure

```
javascript
```



```
// server/config/pricing.js
```

```
module.exports = {
```

```
  plans: {
```

```
    // DevOps Only
```

```
    devops_starter: {
```

```
      name: 'DevOps Starter',
```

```
      price: 2999, // ₹2,999/month
```

```
      features: {
```

```
        pipelines: 5,
```

```
        deployments_per_month: 50,
```

```
        infrastructure_instances: 10,
```

```
        monitoring: true,
```

```
        ai_insights: false
```

```
      },
```

```
      modules: ['devops']
```

```
    },
```

```
    devops_pro: {
```

```
      name: 'DevOps Pro',
```

```
      price: 7999,
```

```
      features: {
```

```
        pipelines: 20,
```

```
        deployments_per_month: 200,
```

```
        infrastructure_instances: 50,
```

```
        monitoring: true,
```

```
        ai_insights: true
```

```
      },
```

```
      modules: ['devops']
```

```
    },
```

```
    // Business Ops Only
```

```
    business_starter: {
```

```
      name: 'Business Starter',
```

```
      price: 2499, // ₹2,499/month
```

```
      features: {
```

```
        automations: 10,
```

```
        executions_per_month: 1000,
```

```
        leads: 500,
```

```
        email_processing: 5000,
```

```
        integrations: 5
```

```
      },
```

```
      modules: ['business']
```

```
    },
```

```
    business_pro: {
```

```
      name: 'Business Pro',
```

```
price: 5999,
features: {
  automations: 50,
  executions_per_month: 10000,
  leads: 5000,
  email_processing: 50000,
  integrations: 20
},
modules: ['business']
},
```

// COMBO PACKAGES (Best Value!)

```
complete_starter: {
  name: 'Alkya Complete Starter',
  price: 4999, // ₹4,999/month (save ₹499)
  features: {
    // DevOps features
    pipelines: 5,
    deployments_per_month: 50,
    infrastructure_instances: 10,
    // Business features
    automations: 10,
    executions_per_month: 1000,
    leads: 500,
    email_processing: 5000,
    // Shared
    monitoring: true,
    ai_insights: true,
    cross_domain_workflows: true
  },
  modules: ['devops', 'business']
},
```

```
complete_pro: {
  name: 'Alkya Complete Pro',
  price: 11999, // ₹11,999/month (save ₹1,999)
  features: {
    // DevOps features
    pipelines: 'unlimited',
    deployments_per_month: 'unlimited',
    infrastructure_instances: 100,
    // Business features
    automations: 'unlimited',
    executions_per_month: 50000,
    leads: 'unlimited',
    email_processing: 'unlimited',
    // Shared
```

```

    monitoring: true,
    ai_insights: true,
    cross_domain_workflows: true,
    priority_support: true,
    custom_integrations: true
  },
  modules: ['devops', 'business']
},

enterprise: {
  name: 'Enterprise',
  price: 'custom',
  features: {
    everything: true,
    white_label: true,
    dedicated_support: true,
    sla: '99.9%',
    custom_deployment: true
  },
  modules: ['devops', 'business', 'custom']
}
};

```

// server/middleware/planLimits.js

```
const pricing = require('../config/pricing');
```

```

async function checkPlanLimit(req, res, next) {
  const user = req.user;
  const plan = pricing.plans[user.plan];

```

// Check based on endpoint

```

if (req.path.includes('/business/automations')) {
  const count = await db.get(
    'SELECT COUNT(*) as count FROM business_automations WHERE user_id = ?',
    [user.id]
  );

  if (plan.features.automations !== 'unlimited' &&
    count.count >= plan.features.automations) {
    return res.status(403).json({
      error: 'Plan limit reached',
      message: `Your ${plan.name} plan allows ${plan.features.automations} automations. Upgrade to create more.`,
      upgrade_url: '/billing/upgrade'
    });
  }
}
}

```

```
next();  
}  
  
module.exports = checkPlanLimit;
```

Billing Page Update

typescript

```
// src/pages/Billing.tsx (add to existing)
```

```
export default function Billing() {  
  const plans = [  
    {  
      category: 'DevOps Only',  
      plans: [  
        {  
          name: 'DevOps Starter',  
          price: '₹2,999',  
          features: ['5 Pipelines', '50 Deployments/mo', '10 Instances'],  
          cta: 'Start Free Trial'  
        },  
        {  
          name: 'DevOps Pro',  
          price: '₹7,999',  
          features: ['20 Pipelines', '200 Deployments/mo', '50 Instances', 'AI Insights'],  
          cta: 'Start Free Trial'  
        }  
      ]  
    },  
    {  
      category: 'Business Ops Only',  
      plans: [  
        {  
          name: 'Business Starter',  
          price: '₹2,499',  
          features: ['10 Automations', '1K Executions/mo', '500 Leads'],  
          cta: 'Start Free Trial'  
        },  
        {  
          name: 'Business Pro',  
          price: '₹5,999',  
          features: ['50 Automations', '10K Executions/mo', '5K Leads'],  
          cta: 'Start Free Trial'  
        }  
      ]  
    },  
    {  
      category: 'Complete Platform ★ Best Value',  
      plans: [  
        {  
          name: 'AIkya Complete Starter',  
          price: '₹4,999',  
          originalPrice: '₹5,498',  
          savings: 'Save ₹499/mo',  
          features: [  

```

```

    '✅ All DevOps Starter features',
    '✅ All Business Starter features',
    '✅ Cross-domain workflows',
    '✅ Unified dashboard'
  ],
  cta: 'Start Free Trial',
  popular: true
},
{
  name: 'AIkya Complete Pro',
  price: '₹11,999',
  originalPrice: '₹13,998',
  savings: 'Save ₹1,999/mo',
  features: [
    '✅ Unlimited everything',
    '✅ Priority support',
    '✅ Custom integrations',
    '✅ Advanced AI'
  ],
  cta: 'Start Free Trial',
  popular: false
}
]
}
];

return (
  <div className="p-6">
    <h1 className="text-3xl font-bold mb-8">Choose Your Plan</h1>

    {plans.map(category => (
      <div key={category.category} className="mb-12">
        <h2 className="text-2xl font-semibold mb-6">{category.category}</h2>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
          {category.plans.map(plan => (
            <PlanCard key={plan.name} plan={plan} />
          ))}
        </div>
      </div>
    ))}

    <div className="mt-12 p-8 bg-gradient-to-r from-blue-50 to-purple-50 rounded-lg">
      <h2 className="text-2xl font-bold mb-4">Enterprise</h2>
      <p className="text-gray-700 mb-4">
        Need custom deployment, white-label, or 100+ team members? Let's talk.
      </p>
      <button className="bg-blue-600 text-white px-6 py-3 rounded-lg">

```

Contact Sales

```
</button>
</div>
</div>
);
}
```

Phase 7: Marketing & Documentation (Week 8)

Update README.md

markdown

Aikya - Complete Automation Platform

Aikya is the ****only platform that automates BOTH your technical infrastructure AND your business operations**** using

What Makes Aikya Different?

Most platforms force you to choose:

- ****DevOps tools**** (GitHub, GitLab) handle code but not business
- ****Business automation**** (Zapier, Make.com) handle workflows but not infrastructure

****Aikya does both.**** Your engineering team and operations team use the same platform.

Features

DevOps Module

-  CI/CD Pipeline Management
-  Multi-Cloud Infrastructure (AWS, GCP, Azure)
-  Automated Deployments
-  Real-time Monitoring
-  Security Scanning

Business Ops Module (NEW!)

-  Email Automation & Classification
-  Lead Management & Scoring
-  Support Ticket Triage
-  CRM Integrations (HubSpot, Salesforce)
-  SMS/WhatsApp Automation

AI-Powered Everything

-  Intelligent classification
-  Predictive analytics
-  Automated decision-making
-  Smart recommendations

Cross-Domain Workflows

- Deploy code → Auto-notify customers
- Production error → Create support ticket + Email customers
- Hot lead → Spin up demo environment
- Customer churn risk → Alert sales team

Quick Start

1. ****Install & Run****

```
```bash
```

```
git clone https://github.com/yourusername/aikya.git
```

```
cd aikya
```



[npm install](#)

[npm run dev](#)

```

2. ****Choose Your Module****

- Using Aikya for DevOps? Start with Pipelines
- Using Aikya for Business? Start with Automations
- Using both? Start with the unified Dashboard

3. ****Build Your First Workflow****

- DevOps: Create a deployment pipeline
- Business: Automate email responses
- Cross-domain: Deploy → Notify customers

Pricing

- ****DevOps Starter****: ₹2,999/month
- ****Business Starter****: ₹2,499/month
- ****Complete Starter****: ₹4,999/month (save ₹499!)
- ****Complete Pro****: ₹11,999/month (save ₹1,999!)
- ****Enterprise****: Custom pricing

****All plans include 14-day free trial. No credit card required.****

Use Cases

For Startups

"One platform for everything. Our dev team uses it for deployments, our ops team uses it for customer management. Game ch
— Tech Startup, 15 employees

For Agencies

"We manage 20+ client projects. Aikya automates our DevOps AND our client communication. Saves us 30 hours/week."
— Digital Agency, 25 employees

For SaaS Companies

"When we deploy new features, Aikya automatically emails all customers. When something breaks, it creates support tickets."
— SaaS Company, 50 employees

Documentation

- [\[Getting Started\]\(docs/getting-started.md\)](#)
- [\[DevOps Module Guide\]\(docs/devops.md\)](#)
- [\[Business Ops Module Guide\]\(docs/business-ops.md\)](#)
- [\[Cross-Domain Workflows\]\(docs/cross-domain.md\)](#)
- [\[API Reference\]\(docs/api.md\)](#)

Roadmap

****Q1 2025****

- ☒ DevOps Module
- ☒ Business Ops Module
- ☒ Cross-domain workflows
- ☐ Mobile app

****Q2 2025****

- ☐ Kubernetes native
- ☐ Advanced AI models
- ☐ WhatsApp Business API
- ☐ Slack/Teams deep integration

****Q3 2025****

- ☐ Multi-region deployments
- ☐ Enterprise SSO
- ☐ White-label option
- ☐ Terraform provider

Contributing

We welcome contributions! See [\[CONTRIBUTING.md\]\(CONTRIBUTING.md\)](#)

Support

-  Email: support@aikya.dev
-  Discord: [\[Join our community\]\(https://discord.gg/aikya\)](https://discord.gg/aikya)
-  Docs: [\[docs.aikya.dev\]\(https://docs.aikya.dev\)](https://docs.aikya.dev)
-  Issues: [\[GitHub Issues\]\(https://github.com/yourusername/aikya/issues\)](https://github.com/yourusername/aikya/issues)

License

See [\[LICENSE\]\(LICENSE\)](#)

Landing Page Copy

Hero Section:

Automate Your Tech Stack AND Your Business
With One AI-Powered Platform

- ✓ DevOps automation (CI/CD, Infrastructure, Monitoring)
- ✓ Business automation (CRM, Email, Support, Sales)
- ✓ Everything connected with AI

[\[Start Free Trial\]](#) [\[Watch Demo\]](#)

Used by 500+ startups, agencies, and SaaS companies

Problem Section:

You're Using Too Many Tools

GitHub/GitLab for DevOps ❌

Zapier for business automation ❌

Datadog for monitoring ❌

HubSpot for CRM ❌

Twilio for messaging ❌

= \$500-2000/month + integration nightmares

↓

Alkya Does It All

One platform, one login, one bill ✅

Everything integrated by default ✅

AI-powered automation everywhere ✅

= \$4,999/month for everything

Solution Section:

For Your Engineering Team

[Screenshot of Pipelines Dashboard]

- ✓ Deploy code with AI-optimized pipelines
- ✓ Manage infrastructure across AWS, GCP, Azure
- ✓ Monitor everything in real-time
- ✓ Let AI predict and prevent issues

For Your Business Team

[Screenshot of Business Dashboard]

- ✓ Automate email responses with AI
- ✓ Score and route leads automatically
- ✓ Triage support tickets intelligently
- ✓ Sync everything to your CRM

The Magic: They Work Together

[Screenshot of Cross-Domain Workflow]

- Deploy new feature → Automatically email customers
- Production error → Auto-create tickets + notify users
- Hot lead → Spin up demo environment instantly
- Customer at-risk → Alert sales team proactively

Use Cases Section:

"We replaced 6 tools with Aikya"

Before: GitHub + Zapier + Datadog + HubSpot + Twilio + AWS Console
Cost: \$1,800/month
Integration headaches: Weekly

After: Just Aikya
Cost: \$4,999/month
Integration headaches: Zero
Time saved: 40 hours/month

— Sarah Chen, CTO at TechFlow (50 employees)

Pricing Section:

Simple Pricing. No Surprises.

| [DevOps Only] | [Business Only] | [Complete Platform] ★ |
|---------------|-----------------|-----------------------|
| ₹2,999/month | ₹2,499/month | ₹4,999/month |
| (Save ₹499!) | | |

- | | | |
|-----------------|------------------|----------------------|
| → All features | → All features | → Everything + |
| → 5 pipelines | → 10 automations | → Unlimited both |
| → 50 deploys/mo | → 1K executions | → Cross-domain magic |
| → 10 instances | → 500 leads | → Priority support |

| | | |
|--|--|---------------------------------|
| [Start Free Trial] [Start Free Trial] [Start Free Trial] | | |
| | | |
| [Complete Pro] | | [Enterprise] |
| ₹11,999/month | | Custom |
| (Save ₹1,999!) | | |
| → Everything unlimited | | → White-label |
| → Advanced AI | | → Custom deployment |
| → Priority support | | → Dedicated support |
| → Custom integrations | | → 99.9% SLA |
| [Start Free Trial] | | [Contact Sales] |

Implementation Timeline Summary

Month 1: Core Integration

- **Week 1:** Database schema + Backend structure
- **Week 2:** API routes + Services
- **Week 3:** Frontend components
- **Week 4:** Navigation + UI integration

Month 2: Features & Testing

- **Week 5-6:** Cross-domain workflows
- **Week 7:** Pricing + Plan limits
- **Week 8:** Documentation + Marketing

Month 3: Launch

- **Week 9:** Beta testing (5-10 users)
- **Week 10:** Bug fixes + Refinement
- **Week 11:** Public launch + Marketing
- **Week 12:** Iterate based on feedback

Revenue Projections

Conservative Scenario

Month 1-2 (Beta): 5 users × ₹0 = ₹0 (free beta)

Month 3: 10 users × ₹4,999 = ₹49,990

Month 4: 20 users × ₹4,999 = ₹99,980

Month 5: 35 users × ₹4,999 = ₹1,74,965

Month 6: 50 users × ₹4,999 = ₹2,49,950

Year 1 Target: 200 users × ₹4,999 = ₹9,99,800/month = ₹1.2 Crore ARR

Optimistic Scenario

Month 3: 20 users = ₹99,980

Month 6: 100 users = ₹4,99,900

Month 12: 500 users = ₹24,99,500/month = ₹3 Crore ARR

Competitive Advantages

vs GitHub + Zapier (Current Standard)

- **Them:** 2 separate tools, manual integration, double cost
- **You:** 1 unified platform, automatic integration, lower total cost

vs Jenkins + Make.com

- **Them:** DevOps only + Business only, no AI, complex setup
- **You:** Both integrated, AI everywhere, simple setup

vs Building In-House

- **Them:** 12-18 months development, \$200K+ cost, ongoing maintenance
 - **You:** Ready now, \$5K/month, we maintain it
-

Next Steps for You

This Week

1. ☒ Review this integration plan
2. ☒ Decide: Full integration OR separate products?
3. ☒ Choose starting point: Add Business to AIkya OR sell separately?

Next Week

4. Start Phase 1 (Database + Backend)
5. Build first cross-domain workflow
6. Create demo video

This Month

7. Complete core integration
 8. Find 5 beta testers
 9. Launch beta program
-

Questions for You

To finalize the plan, I need to know:

1. **Current AIkya Status?**
 - How many users currently?
 - Revenue (if any)?
 - Is it live in production?
2. **Your Goal?**
 - Build a startup (raise funding)?
 - Generate revenue (bootstrap)?
 - Portfolio project?
3. **Resources?**
 - Solo or team?
 - Full-time or side project?
 - Budget for tools/marketing?

4. Technical Preferences?

- Keep SQLite or migrate to PostgreSQL?
- Deploy where (Vercel, AWS, DigitalOcean)?
- Any tech constraints?

5. Integration Approach?

- Full integration (Model 3 - Unified Product) ★
 - Separate modules (Model 2 - AIkya + Add-on)
 - Sequential (Model 1 - Use AIkya as backend)
-

My Recommendation

Go with Model 3: Unified Product

Why?

1. Unique market position (no competitor does both)
2. Higher revenue per customer
3. Lower churn (stickier product)
4. Easier to sell ("one platform for everything")
5. Leverages your existing infrastructure (80% already built!)

Timeline: 3 months to launch Investment: Minimal (leverage what you have) Revenue Potential: ₹10L+/month within 6 months

Ready to start? Tell me:

- Which phase should we detail first?
- Any specific concerns?
- What's your immediate next step?

Let's build this! 🚀 : [{ type: 'send_email', params: { to: customer.email, subject: New features deployed:
\${project_name} v\${version} }, body: ` Hi there!

We just deployed new features to \${project_name}:

```
${features.map(f => `• ${f}`).join('\n')}
```

Check it out: <https://app.example.com>

```
,  
  }  
}]  
});  
}
```

```
}
```

```
// When Business gets hot lead, auto-create demo environment static async onHotLead(lead) { if (lead.score >=  
80) { // Use DevOps module to spin up demo environment const demoEnv = await  
InfrastructureService.createDemoEnvironment({ name: demo-${lead.id}, template: 'standard', ttl: 7 * 24 * 60 *  
60 // 7 days });
```

```
// Use Business module to send email with demo link
```

```
await WorkflowEngine.execute({  
  actions
```